

CS69021: Computing Lab-1

Test-7 || August 24, 2026

Time: 2.25 hours

===== Instructions =====

1. Q1 carries **30 marks**, Q2 carries **30 marks**, and Q3 carries **40marks**.
2. In the case of user input, assume only valid values will be passed as input.
3. You can use C or C++ as the programming language.
4. **You are allowed to use STL libraries in C++.**
5. Submission Guidelines
 - a. Create a separate C/C++ file for each sub-question, named as:
 <rollno_seatno>_Q1.cpp,<rollno_seatno>_Q2.cpp,
 <rollno_seatno>_Q3.cpp
 - b. Compress all files into a single zip named:
 <rollno_seatno>_T7.zip
 - c. Incorrect input/output formatting or incorrect file naming will result in a **75% penalty**.
 - d. Submit the zip file on Moodle.
 - e. **Example:** For roll number **26CS60R15 seated at seat no 10:**
 Files:26CS60R15_10_Q1.cpp,26CS60R15_10_Q2.cpp,26CS60R15_10_Q3.cpp
 Zip: 26CS60R15_10_T7.zip
6. Hardcoding outputs would yield 0 marks.
7. **Plagiarism in any form is strictly prohibited. If detected, it may result in a significant grade penalty.**

=====

Q1) The Journey Back Home [30 Marks]

You are a student staying at **MMM Hall of Residence, IIT Kharagpur**. One evening, you need to travel to the **B. C. Roy Technology Hospital**.

For this problem, assume that the IIT Kharagpur campus can be represented as a **grid-like network of roads**, where roads intersect at junctions at right angles.

However, some roads are **blocked**, and you do not know which route will lead to the hospital. Therefore, at every junction, you ask a stranger for directions. The stranger can only tell you which direction to take to reach the **next junction**.

The possible directions are:

- N — North
- S — South
- E — East
- W — West

Many of the strangers may not know the correct route and may **misguide you**, causing you to take unnecessary detours or revisit locations that you have already visited.

After asking **N strangers**, you finally reach **B. C. Roy Technology Hospital**.

At the end of the treatment, you now need to return to **MMM Hall of Residence**. Fortunately, you remember the complete sequence of directions that you followed while travelling from MMM Hall to the hospital. **However, you do not know the paths which you have not visited before**, so you have no choice but to avoid them.

Your task is to determine and print the **shortest known path that will take you back from the hospital to MMM Hall**, using the information available from your journey.

The Task ~

Given a sequence of directions representing the journey from **MMM Hall of Residence to B. C. Roy Technology Hospital**, find the **shortest path back to MMM Hall of Residence**.

The journey may contain unnecessary movements and detours because some strangers may have provided incorrect directions.

You must print the **shortest sequence of directions in a space-separated manner** that takes you from the final position back to the starting position.

Input Format

- The first line contains a single integer N , representing the total number of directions you followed on your way to the hospital.
- The second line contains N space-separated characters (from the set N, S, E, W), representing the exact sequence of your journey.

Output Format

- Print a single line containing a space-separated sequence of characters representing the shortest known path back to MMM Hall of Residence.

Constraints

- $1 \leq N < 10000$
 - The directions are strictly uppercase letters: N, S, E, W.
 - **Note:** In case there is more than one shortest path possible, print any one of the possible paths.
-

Examples ~

Example 1

Input:

13

W N W N E S W W N W S W N

Output:

S E N E S E E S E

Explanation

The given sequence represents the journey from **MMM Hall of Residence** to **B. C. Roy Technology Hospital**.

The required output is therefore the **shortest known sequence of directions after deleting all cycles and redundant moves** that takes you from the hospital back to MMM Hall.

Q2) Escape from Tata Sports Complex [30 Marks]

You are in a hurry to get to your class. Due to heavy traffic on the roads, you decide to take a shortcut by cycling through the Tata Sports Complex (TSC) ground, which is represented as a 2D grid of size $H \times W$ (H rows and W columns).

However, there are N **guards** present inside the ground. Each guard has a different level of fitness and can cover an area around themselves with a certain **radius** R . Here, radius R means the maximum number of hops a guard can make, and a guard can move in any of the **8 directions** (including diagonals) per hop. If you enter any cell that a guard can reach within their radius R , you will be caught.

You are given the positions of all the guards and the radius within which each guard can catch you.

You enter the ground at a specific starting cell in the **first column**, and need to reach any of the K **exit gates** located in the **last column**. You can cycle to any of your 8 adjacent cells (horizontal, vertical, and diagonal).

If there exists a path through the ground such that you can reach any exit gate without coming within the catching radius of any guard, you are safe. Otherwise, it is impossible to cross the ground without being caught.

The Task: If such a path exists, print **YES** followed by a space and the **minimum number of steps** required to reach an exit gate. If no such path exists, print **NO**.

Constraints

- $1 \leq H, W \leq 1000$
- $1 \leq N \leq 10^5$
- $1 \leq K \leq H$
- $0 \leq R \leq 1000$
- The starting coordinate (S_r, S_c) will always be in the first column, meaning $S_c = 0$.

- Every exit gate coordinate (E_r, E_c) will always be in the last column, meaning $E_c = W - 1$.
- All guard coordinates (r, c) will satisfy $0 \leq r < H$ and $0 \leq c < W$.

Input Format

- The first line contains four space-separated integers: H (rows), W (columns), N (number of guards), and K (number of exit gates).
- The second line contains two space-separated integers: S_r and S_c , representing your starting row and column.
- The next K lines each contain two space-separated integers: E_r and E_c , representing the row and column of an exit gate.
- The next N lines each contain three space-separated integers: row, col and R , representing the row, column, and catching radius of a guard.

Output Format

- Print YES and the minimum distance if a safe path exists; otherwise, print NO.

Sample Test Cases

Sample Input 1

5 5 1 2

2 0

0 4

4 4

2 2 1

Sample Output 1

YES 5

Explanation for Sample 1:

The grid has 5 rows and 5 columns. You start at row 2, column 0: (2, 0). You can exit at either (0, 4) or (4, 4). There is 1 guard located at (2, 2) with a hop radius of $R = 1$. Because the guard can hop in any of the 8 directions, their catching area expands into a 3x3 square block. They will catch you if you step on (2, 2) or any of its immediate neighbors: (1,1), (1,2), (1,3), (2,1), (2,3), (3,1), (3,2), (3,3).

You are safe because you can use your own 8-directional movement to cycle around this block. To reach the exit at (0, 4) in the minimum number of steps, you can take a diagonal shortcut to the top edge and move across:

(2,0) -> (1,0) -> (0,1) -> (0,2) -> (0,3) -> (0,4).

This takes exactly 5 steps. Since you reached an exit gate safely, the output is YES 5.

Sample Input 2

5 5 3 1

2 0

2 4

0 2 1

2 2 1

4 2 1

Sample Output 2

NO

Explanation for Sample 2: The grid has 5 rows and 5 columns. You start at (2, 0) and there is only 1 exit gate at (2, 4). There are 3 guards:

Guard 1 at (0, 2) with $R = 1$ guards columns 1 to 3 in rows 0 to 1.

Guard 2 at (2, 2) with R = 1 guards columns 1 to 3 in rows 1 to 3.

Guard 3 at (4, 2) with R = 1 guards columns 1 to 3 in rows 3 to 4.

Because each guard covers a 3x3 square block, their areas combine to form a solid vertical wall 3 columns thick (covering all cells in columns 1, 2, and 3) from the top row to the bottom row. Since you start on the left side of this wall at column 0, and your only exit is on the right side at column 4, it is entirely impossible to bypass the wall without entering a guarded cell. Therefore, the output is NO.

Q3) Class Schedule Walk [40 Marks]

You are a student at **IIT Kharagpur**, moving between different buildings on campus to attend a fixed sequence of classes. The campus buildings are connected by walkways, and each walkway takes a known amount of time to cross.

You always take the **fastest possible route** to your next class. If you reach a class on time, you attend it. If you are late, the professor does not allow you to enter, and you immediately leave for the next class.

Your task is to report the **arrival time at each class**.

Campus Layout ~

The campus is represented as an **undirected, connected, weighted graph** with V buildings numbered from 0 to $V-1$ and E walkways.

Each walkway:

- Connects two distinct buildings.
- Takes w ticks to cross, where $1 \leq w \leq 10000$.
- There is at most one walkway between any pair of buildings (no multi-edges).
- No walkway connects a building to itself.

Time is measured in integer ticks starting from $t = 0$.

You begin at building S at tick 0.

Classes ~

You must attend C classes in the **given order**.

Class i ($1 \leq i \leq C$) is described by three integers:

loc_i $start_i$ dur_i

where:

- loc_i — the building where class i is held.
- $start_i$ — the tick at which class i begins.
- dur_i — the duration of class i .

The class ends at: **$start_i + dur_i$**

Movement and Timing Rules ~

You process the classes **strictly in order** (The schedule is guaranteed to be **chronologically sorted** by the **start_i**).

For each class *i*, you travel from your **current building** to **loc_i** using a path having the **minimum possible total weight**.

Your arrival time is:

$\text{arrival_time} = \text{current_time} + \text{shortest_path_distance}(\text{current_building}, \text{loc}_i)$

At **loc_i**, exactly one of the following happens:

On Time If: $\text{arrival_time} \leq \text{start}_i$

you are allowed to attend the class.

- If you arrive early, you wait until the class begins.
- Record your **arrival time** as an answer for class *i*.
- You leave for the next class at: **start_i + dur_i**

Late If: $\text{arrival_time} > \text{start}_i$

you are not allowed to attend the class.

- Record -1 as an answer for class *i*.
- You do **not** wait.
- You leave immediately for the next class at your **arrival time**.
- Your current building becomes **loc_i**.

Either way, set **current_building = loc_i** before moving to class *i*+1. (You are physically present at **loc_i** regardless of whether you attended.)

Note: it's possible to be marked late for class *i* and still arrive on time for class *i*+1, including when **loc_i = loc_{i+1}** and **start_{i+1} = arrival_time** for class *i* (zero travel distance, boundary equality).

For the first class, you start from building S at tick 0.

The Task ~

Given the campus graph, your starting building, and the complete class schedule, print the **arrival time for every class**.

For class *i*:

- Print the tick at which you arrive at loc_i , if you arrive on time and attend the class.
- Print -1 if you arrive after $start_i$ and therefore miss the class.

Print all C values as **space-separated integers on a single line**.

Input Format ~

```
V E
u v w
u v w
...
S
C
loc_1 start_1 dur_1
loc_2 start_2 dur_2
...
loc_C start_C dur_C
```

The first line contains V and E.

The next E lines describe the walkways. Each line contains:

```
u v w
```

where u and v are the connected buildings and w is the time required to cross the walkway.

The next line contains the starting building S.

The following line contains the number of classes C.

The next C lines contain the details of the classes in the order in which they must be visited.

Constraints

```
2 ≤ V ≤ 2000
V-1 ≤ E ≤ 15000
1 ≤ w ≤ 10000
1 ≤ C ≤ 50
0 ≤ S, loc_i ≤ V-1
0 ≤ start_i, dur_i ≤ 100000
```

Additional guarantees:

1. The graph is connected, with no self-loops and no multiple edges between the same pair of buildings.
 2. The class schedule (`loc_i`, `start_i`, `dur_i`) appears in sorted order with respect to `start_i` (chronologically increasing) .
 3. All input values are integers, and all intermediate arrival times fit comfortably within standard 32-bit integer range.
-

Example ~

Input

```
5 5
0 1 2
1 2 3
0 3 1
3 4 1
4 2 2
0
3
2 6 4
4 20 5
1 8 3
```

Output

```
4 12 -1
```

Explanation

Class 1: `loc = 2`, `start = 6`, `dur = 4`

You start from building 0 at tick 0.

There are two relevant routes from 0 to 2:

$$0 \rightarrow 1 \rightarrow 2 = 2 + 3 = 5$$

$$0 \rightarrow 3 \rightarrow 4 \rightarrow 2 = 1 + 1 + 2 = 4$$

Therefore, the shortest distance is 4.

You arrive at:

$$0 + 4 = 4$$

Since $4 \leq 6$, you attend the class.

Record:

4

The class ends at:

$$6 + 4 = 10$$

So you leave building 2 at tick 10.

Class 2: loc = 4, start = 20, dur = 5

The shortest distance from building 2 to building 4 is 2.

Therefore, you arrive at:

$$10 + 2 = 12$$

Since $12 \leq 20$, you attend the class.

Record:

12

The class ends at:

$$20 + 5 = 25$$

So you leave building 4 at tick 25.

Class 3: loc = 1, start = 8, dur = 3

The shortest route from building 4 to building 1 is:

$$4 \rightarrow 3 \rightarrow 0 \rightarrow 1$$

with total distance:

$$1 + 1 + 2 = 4$$

You therefore arrive at:

$$25 + 4 = 29$$

Since:

$$29 > 8$$

you are late and cannot attend the class.

Record:

$$-1$$

Therefore, the final output is:

$$4 \ 12 \ -1$$
